



African Leadership University
Kigali Innovation City, Free Economic Zone, Gasabo, Kigali

Modeling Human Activity States Using Hidden Markov Models

Formative 2: Hidden Markov Models

By

Principie Cyubahiro

p.cyubahiro@alustudent.com
BSc Software Engineering
Machine Learning Techniques II
African Leadership University, Kigali

October 2026

Abstract

In this report, four explicit human activities such as Standing, Walking, Jumping and Still states are modeled as hidden activities behind phone motion signals. I recorded 50 labelled files (with 440.6 s total) at 50 and 100 Hz using Sensor Logger on an iPhone 14 Pro, harmonized all signals to 50 Hz and divided the signals into 2 s windows. The features for each window are, along with a few other, seven time-domain features, three FFT (fast Fourier transform) ones, all log-transformed if necessary and Z-score normalized. The same is coded from scratch in NumPy and trained using the Baum-Welch algorithm until reaching a log likelihood increase of less than 1e-4 from the labels, and then is decoded using the Viterbi algorithm. Every state achieved 1.000 sensitivity, specificity and accuracy on two unseen test files consisting of eight held-out recordings (68 windows). The limit of the model illustrated by a noise test occurs when features become noisy and overconfident emissions take precedence, in which case the transition matrix makes a very small contribution.

Keywords: human activity recognition, Hidden Markov Model, Baum-Welch, Viterbi, accelerometer, gyroscope, FFT features.

Contents

1	Background and Motivation	3
2	Data Collection and Pre-processing	3
3	HMM Setup and Implementation	4
4	Results and Interpretation	4
5	Discussion and Conclusion	5

1. Background and Motivation

The phones already have an accelerometer and gyroscope, but they are never labeled walking or still: the activity is masked by noisy measurements. Doing activity monitoring for the elderly and patients recovering at home at low cost but again with the commonality of the smartphone devices in the developing country of Rwanda. If someone has been inactive for too long, or becomes extremely active all of a sudden, it means a member of a family or community health worker could check in early. The reason a Hidden Markov Model is useful is that the signal pattern obtained by a particular activity (emissions) is typical and activities are continuous over time and have a meaningful order (transitions).

2. Data Collection and Pre-processing

2.1 Collection

I recorded all the data myself on 28th October 2026 using Sensor Logger app, logging the accelerometer, gyroscope and gravity sensors. The phone was held at waist level for Standing, Walking and Jumping, and placed flat on a table for Still. Two sessions were recorded at different rates.

User	Device and app	Sampling rates recorded	Harmonized rate
Principie Cyubahiro	iPhone 14 Pro, Sensor Logger 1.68	50 Hz and 100 Hz (measured from timestamps)	50 Hz

Activity	Files	50 Hz / 100 Hz files	Total duration	File length
Standing	10	5 / 5	97.0 s	7.0 to 10.0 s
Walking	10	5 / 5	100.0 s	10.0 s
Jumping	20	14 /6	149.1 s	5.0 to 10.0 s
Still	10	6 / 4	94.5 s	7.0 to 10.0 s
Total	**50**	**30 / 20**	**440.6 s**	**5.0 to 10.0 s**

Video evidence. One video per activity (Standing, Walking, Jumping, Still) shows my full body and face while I hold the phone and record. All four are in one folder: **Principie Cyubahiro_Formative 2 Video Evidence**

Github link: https://github.com/PrincipieCyupe/formative2-hmm-activity-recognition

2.2 Pre-processing

- **Extraction and checks;** There is one ZIP in each activity and both were unzipped. The rates were taken from the nanosecond time stamps and revealed that the two Standing folders were incorrectly labeled. The largest difference between samples in any file is 20ms, so no dropouts.
- **Different sampling rates;** After the 100 Hz files go through a 4th order Butterworth low pass filter at 20 Hz to eliminate aliasing, each file is resampled onto a uniform 50 Hz grid. Instead of 60 Hertz, I opted for 50 because it is the lower common rate; below 20 Hz is human motion, the filter reduces aliasing before resampling.
- **Cleaning;** 0.25 s is trimmed at both ends to remove the tap on the start and stop button, and files are capped at 10 s. Each recording is stored as a labelled csv file with a timestamp column and all 50 recordings are exported to a single table of 22,031 rows with the timestamp column.
- **Windowing**; 2 s windows (100 samples at 50 Hz) with 50% overlap, giving 381 windows. My step and jumps are 1.5-2.5 Hz, meaning a window contains 3 or more cycles and those cycles are separated by 0.5 Hz in the FFT resolution. All the files have a common rate, 100 samples always represents 2 s.

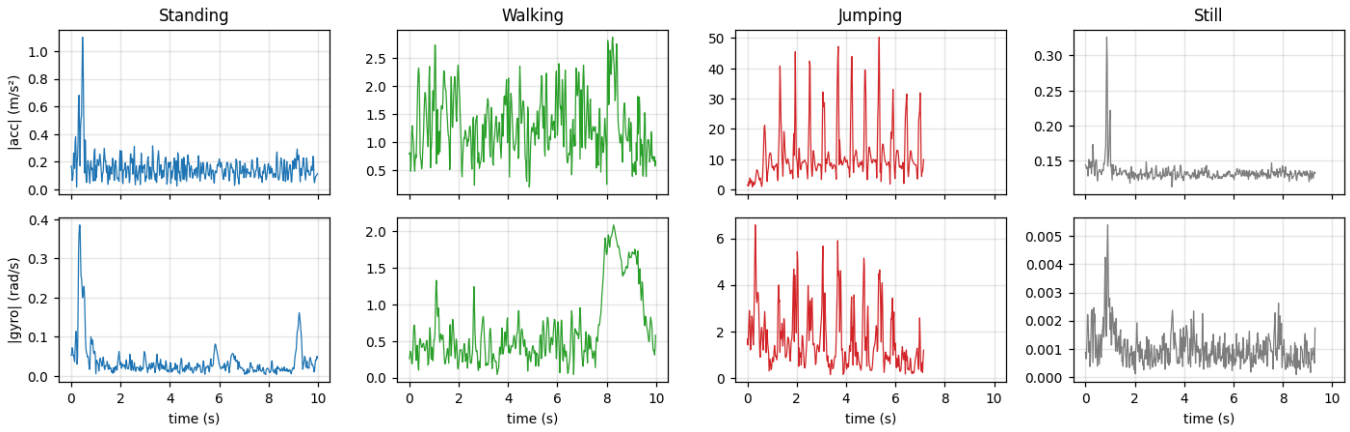


Figure 1. One harmonized recording per activity: accelerometer magnitude (top) and gyroscope magnitude (bottom).

3. HMM Setup and Implementation

3.1 Features and normalization

Feature	Domain	Why it separates the activities
acc_std, acc_rms, acc_sma	Time	Spread, energy and magnitude area of body acceleration. RMS: Still 0.13, Standing 0.17, Walking 1.70, Jumping 11.94 m/s².
gyr_rms, gyr_std	Time	Rotation intensity: a held phone sways, a phone on a table does not (0.043 vs 0.002 rad/s).
corr_yz	Time	Coupling of vertical and forward axes, strongest for Jumping (0.72).
tilt_deg	Time	Phone angle to gravity: 2.9° on a table, 10.6° standing, 34° walking.
dom_freq	FFT	Rhythm of motion: 2.0 Hz step rate when walking, no clear peak when still.
spec_energy	FFT	Total spectral power, intensity seen in frequency.
spec_entropy	FFT	Low for a clean rhythm, highest for flat noise of Still (0.88).

The six intensity features cover three orders of magnitude, hence they are log-transformed to emphasize the Gaussian nature they are assumed to have by the emission model. All these 10 are then standardized using Z-score normalization, so features with larger scales do not dominate the likelihood. These are used based on the training windows only, so no test information will leak in.

3.2 Model components and algorithms

Hidden states Z	Observations X	Transitions A	Emissions B	Initial pi
Standing, Walking, Jumping, Still	10-dim normalized feature vector per 2 s window	4 x 4, P(next activity | current), one step = 1 s	Gaussian per state, diagonal covariance, variance floor 0.01	P(first window is each activity)

The model is implemented from scratch in pure NumPy, entirely in log space avoiding underflow. **Forward-Backward** returns posteriors and likelihood of the sequence per window. **Baum-Welch** learns pi, A and the Gaussians; initially these are estimated from the labelled windows, a prior of 1 is added to the transition counts to ensure it is possible to have a transition even after its count is 0, and is terminated when the mean log-likelihood is improved by at most 1e-4. Viterbi returns the most likely set of activities. All the metrics are measured on the Viterbi path, which is on whom the algorithm is evaluated.

For training, 42 recordings were chained together to be played day-by-day (phone on a table, pick it up and stand, walk, then jump, 313 windows total). After 5 iterations, Baum-Welch converged and the mean log-likelihood went from 1.566 to 2.021. Before Baum-Welch, the accuracy was 0.987 for the training set and after it, there was a tiny drop to 0.984, which is low as it maximizes likelihood and not label accuracy.

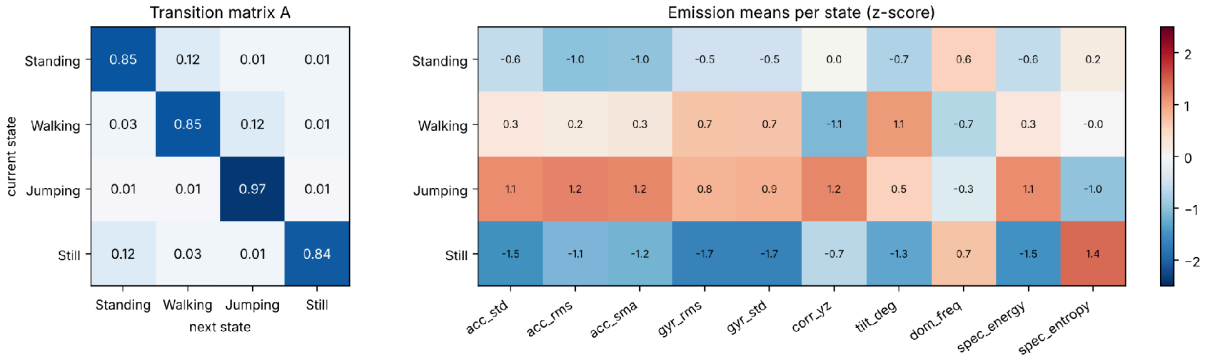


Figure 2. Learned transition matrix (left) and emission means per state in Z-score units (right).

4. Results and Interpretation

4.1 Unseen test data

For each activity I held out one 50 Hz and one 100 Hz recording, eight in total. These were not used for training, nor for normalization. I created two test sequences from eight held-out recordings: test_sequence_1.csv (originally recorded at 50 Hz, with activities ordered as Still, Standing, Walking, Jumping) and test_sequence_2.csv (originally recorded at 100 Hz, with activities ordered as Jumping, Walking, Standing, Still).

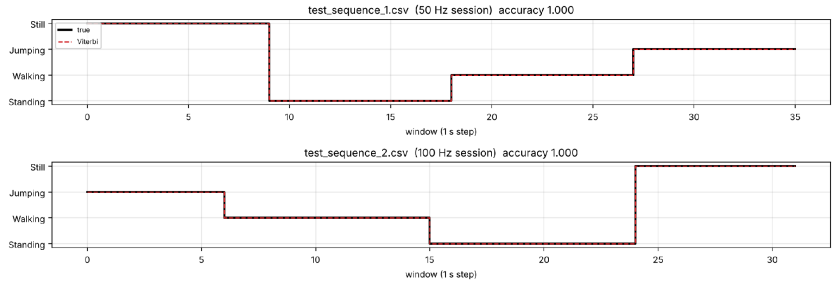


Figure 3. True activity (black) and Viterbi decoding (red) for both unseen test files.

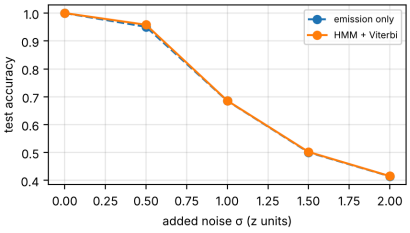


Figure 4. Test accuracy when Gaussian noise is added to the features.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Standing	18	1.000	1.000	1.000
Walking	18	1.000	1.000	1.000
Jumping	15	1.000	1.000	1.000
Still	17	1.000	1.000	1.000
All windows	**68**	**1.000**	**1.000**	**1.000**

Table 1. Per-state metrics on unseen data (window level).

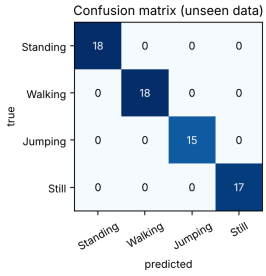


Figure 5. Confusion matrix on unseen data.

All 68 unseen windows are correct, no flickering and no incorrectly predicted switches. Test file 2 has a high probability of transition that can be as low as 0.03, yet it is still obtained correctly: a high transition probability is logged away while the evidence of the transition is weak. Both 50 Hz and 100 Hz indicate a score of 1.000, and harmonizing the rates was successful. When estimating emission only, there was also a 1.000 chance when predicting the transitions, which indicates that the transitions are not yet working as done on clean data.

5. Discussion and Conclusion

Generalization; The model generalizes very well and even to new recordings of mine, taken from the same person, phone and date, but if I'm not being meticulous with the perfect score, that's 68 windows from the same person, phone and day that the features separate well, not necessarily from a different person or phone position.

Easiest and hardest activities; Jumping is the easiest, and it has some 7 times the acceleration RMS of walking Standing vs Still is the hardest: their accelerometer values almost overlap, and all three Still errors in training were read as Standing. The only thing that divides them is their gyroscopic sway and tilt, hence the inclusion of these features.

Transition probabilities; This range of 0.84 to 0.97 is realistic because people do not change activity every second. Jumping is stickier because the jumping activity periods were longest. The forward links (Still to Standing, Standing to Walking, Walking to Jumping) are as described in my scripted routine with 0.12 each because each session started on the table, and pi means 0.75 because they were all on the table from the start. So A reflects my collection protocol more than a real day, which would have more Standing and Walking back and forth.

Noise and sampling rate; Measuring rates from swapped Standing labels captures the rates and after filtering and resampling the rate has no measurable effect. However important is the simulated sensor noise: For sigma = 1, accuracy becomes 0.69, while for sigma = 0.5, Viterbi outperforms emission-only at 0.958, and not at all at 0.951. The difference in emission log-likelihood between the best and second-best is 102.7 nats, and the transition penalty is 3.3 nats, which means that the correlated features are treated as independent evidence, and the emissions are overconfident.

Improvements; Test other phone positions, other recordings from different people (for example other students in the same class), a longer unscripted and more natural transition model (less decorrelated features or full covariance, more emphasis from the transition model).

Conclusion; A model trained from scratch with Baum-Welch, and decoded with Viterbi, using all four activities from Gaussian HMMs built in NumPy, achieved perfect sensitivity, specificity and accuracy on unseen recordings across all four states. The robustness question is one requiring other people and noisier data which would allow the transition model to play a larger role with better calibrated emissions.